

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проектированию
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Реализация алгоритма поиска наибольшего паросочетания»

25.12.25
хорошо
фид

Выполнил:
студент группы 24BBB3
Алмакаев В.В.
Принял:
Юрова О.В.

Пенза 2025

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет Вычислительной техники

Кафедра "Вычислительная техника"

"УТВЕРЖДАЮ"

Зав. кафедрой ВТ

« ____ » ____ 20__

ЗАДАНИЕ

на курсовое проектирование по курсу

«Логика и основы алгоритмизации в инженерных задачах»
Студенту Аннакову Владимиру Викторовичу Группа 24ВВВЗ
Тема проекта «Реализация алгоритма поиска наибольшего
паросочетания»

Исходные данные (технические требования) на проектирование

- Разработка алгоритмов и программного обеспечения в
соответствии с формой задания курсового проекта.
Решительная записка должна содержать:
1. Постановку задачи
 2. Теоретическую часть задания
 3. Описание алгоритма поставленной задачи
 4. Пример ручного расчета задания и вычислений (на небольшом
участке работы алгоритма)
 5. Описание самой программы
 6. Тесты
 7. Список литературы
 8. Листинг программы
 9. Результат работы программы;

Объем работы по курсу

1. Расчетная часть

Ручной расчет работы алгоритма

2. Графическая часть

Схема алгоритма в формате блок-схем

3. Экспериментальная часть

Тестирование программы

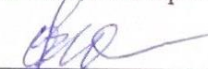
Результаты работ программы на тестовых данных

Срок выполнения проекта по разделам

- 1 Исследование теоретической части курсового проекта
- 2 Разработка алгоритмов программы
- 3 Разработка программы
- 4 Тестирование и завершение разработки программы
- 5 Оформление пояснительной записки
- 6
- 7
- 8

Дата выдачи задания "12" 09. 2025

Дата защиты проекта " " "

Руководитель Юрова О.В. 

Задание получил "12" сентября 2025 г.

Студент Алмаков В.В. 

Содержание

Реферат	4
Введение	5
1. Постановка задачи	7
2. Теоретическая часть задания	8
3. Описание алгоритма программы.....	11
4. Описание программы.....	16
5. Тестирование.....	22
Заключение.....	29
Список литературы	30
Приложение А.....	31

Реферат

Отчет 32 страницы, 13 рисунков, 1 таблица, 3 источника.

ДВУДОЛЬНЫЙ ГРАФ, ТЕОРИЯ ГРАФОВ, ПАРОСОЧЕТАНИЕ, АЛГО- РИТМ КУНА.

Цель исследования – разработка программы, способной определять наибольшее паросочетание, используя алгоритм Куна.

В работе рассмотрен алгоритм Куна, на основе которого находится наибольшее паросочетание двудольного графа. При помощи данного алгоритма возможно решение задачи о назначениях.

Введение

Граф — это абстрактный математический объект, который представляет собой множество вершин графа и набор рёбер, которые соединяют вершины между собой. Например, за множество вершин можно взять множество населенных пунктов, а в качестве рёбер представить дороги, соединяющие эти пункты.

Графы могут различаться направленностью, ограничениями на количество связей и дополнительными данными о вершинах или рёбрах. Многие структуры могут быть представлены графами. Например, строение компьютерных сетей можно смоделировать при помощи ориентированного графа, в котором вершины — это компьютеры, а дуги (ориентированные рёбра) — соединяющие их сети.

Различают несколько видов графов.

Граф, или неориентированный граф G — это упорядоченная пара $G = (V, E)$, где V — это непустое множество вершин или узлов, а E — множество пар (в случае неориентированного графа — неупорядоченных) вершин, называемых рёбрами. Ориентированный граф (сокращённо орграф) G — это упорядоченная пара $G = (V, A)$, где V — непустое множество вершин или узлов, и A — множество (упорядоченных) пар различных вершин, называемых ориентированными рёбрами (дугами). Смешанный граф G — это граф, в котором некоторые рёбра могут быть ориентированными, а некоторые — неориентированными. Записывается упорядоченной тройкой $G = (V, E, A)$, где V , E и A определены так же, как выше.

Двудольный граф или биграф — это граф, множество вершин которого можно разбить на две части таким образом, что каждое ребро графа соединяет какую-то вершину из одной части с какой-то вершиной другой части, то есть не существует ребра, соединяющего две вершины из одной и той же части.

В данной курсовой работе будет рассматриваться двудольный граф.

Паросочетанием в теории графов называют множество попарно несмежных рёбер, то есть рёбер, не имеющих общих вершин.

Выделяют несколько видов паросочетаний.

Максимальное паросочетание — это такое паросочетание M в графе G , которое не содержится ни в каком другом паросочетании этого графа, то есть к нему невозможно добавить ни одно ребро, которое бы являлось несмежным ко всем рёбрам паросочетания. Другими словами, паросочетание M графа G является максимальным, если любое ребро в G имеет непустое пересечение, по крайней мере, с одним ребром из M .

Наибольшее паросочетание (или максимальное по размеру паросочетание) — это такое паросочетание, которое содержит максимальное количество рёбер. У графа может быть множество наибольших паросочетаний. При этом любое наибольшее паросочетание является максимальным, но не любое максимальное будет наибольшим.

Для нахождения наибольшего паросочетания можно использовать алгоритм Куна. Этот алгоритм можно описать так: сначала возьмём пустое паросочетание, а потом — пока в графе удаётся найти увеличивающую цепь, — будем выполнять чередование паросочетания вдоль этой цепи, и повторять процесс поиска увеличивающей цепи. Как только такую цепь найти не удалось — процесс останавливаем, — текущее паросочетание и есть максимальное.

1. Постановка задачи

Исходный граф в программе должен задаваться матрицей смежности, причём при генерации данных должны быть предусмотрены граничные условия.

Программа должна работать так, чтобы пользователь вводил количество вершин для генерации матрицы смежности двудольного графа. После ввода количества вершин, пользователь может выбрать как заполнить граф – вручную или автоматически. После заполнения матрицы он может вызвать функцию алгоритма Куна, которая выведет на экран наибольшее паросочетание. Необходимо предусмотреть различные исходы поиска, чтобы программа не выдавала ошибок и работала правильно.

Устройство ввода – клавиатура и мышь.

Задания выполняются в соответствии с вариантом №27.

2. Теоретическая часть задания

Граф $G = (V, X)$ (рисунок 1) называется *двудольным*, если множество V его вершин допускает разбиение на два непересекающихся подмножества V_1 и V_2 (две доли), причем каждое ребро графа соединяет вершины из разных долей.

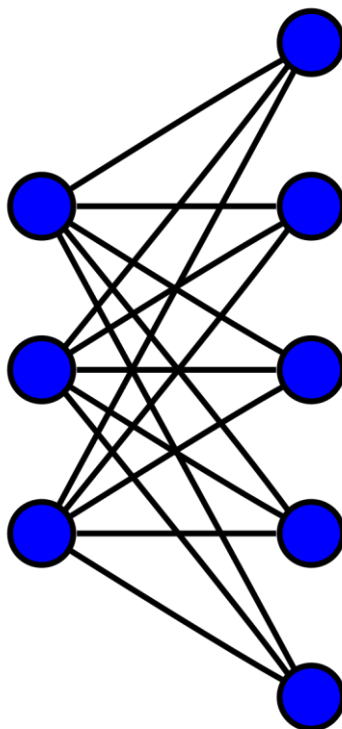


Рисунок 1 – двудольный граф

Обозначим $G = (V_1, V_2, X)$ двудольный граф G с долями V_1 и V_2 . *Паросочетанием* P для двудольного графа $G = (V_1, V_2, X)$ называется любое множество попарно несмежных ребер в G .

Цепью длины k назовём некоторый простой путь (т.е. не содержащий повторяющихся вершин или рёбер), содержащий k рёбер.

Чередующейся цепью (в двудольном графе, относительно некоторого паросочетания) назовём цепь, в которой рёбра поочередно принадлежат/не принадлежат паросочетанию.

Увеличивающей цепью (в двудольном графе, относительно некоторого паросочетания) назовём чередующуюся цепь, у которой начальная и конечная вершины не принадлежат паросочетанию.

На основании предыдущих терминов можно сформулировать теорему Бержа.

Теорема Бержа:

Паросочетание является максимальным тогда и только тогда, когда не существует увеличивающих относительно него цепей.

Алгоритм Куна — непосредственное применение теоремы Бержа. Его можно кратко описать так: сначала возьмём пустое паросочетание, а потом — пока в графе удаётся найти увеличивающую цепь, — будем выполнять чередование паросочетания вдоль этой цепи, и повторять процесс поиска увеличивающей цепи. Как только такую цепь найти не удалось — процесс останавливаем, — текущее паросочетание и есть максимальное.

Алгоритм Куна — просто ищет любую из таких цепей с помощью **обхода в глубину** или **в ширину**. Он просматривает все вершины графа по очереди, запуская из каждой обход, пытающийся найти увеличивающую цепь, начинающуюся в этой вершине. Удобнее описывать этот алгоритм, считая, что граф уже разбит на две доли.

Алгоритм просматривает все вершины v первой доли графа: $v = 1 \dots n_1$. Если текущая вершина v уже насыщена текущим паросочетанием (т.е. уже выбрано какое-то смежное ей ребро), то эту вершину алгоритм пропускает. Иначе — алгоритм пытается насытить эту вершину, для чего запускается поиск увеличивающей цепи, начинающейся с этой вершины.

Поиск увеличивающей цепи осуществляется с помощью обхода в глубину. Изначально обход в глубину стоит в текущей ненасыщенной вершине v первой доли. Просматривая все рёбра из этой вершины, пусть текущее ребро — это ребро (v, to) . Если вершина to ещё не насыщена паросочетанием, то, значит, алгоритм смог найти увеличивающую цепь: она состоит из единственного ребра (v, to) ; в таком случае он включает это ребро в паросочетание

и прекращает поиск увеличивающей цепи из вершины v . Иначе, — если to уже насыщена каким-то ребром (p, to) , то алгоритм попытается пройти вдоль этого ребра: тем самым попробует найти увеличивающую цепь, проходящую через рёбра (v, to) , (to, p) . Для этого просто перейдёт в обходе в вершину p — и теперь уже пробует найти увеличивающую цепь из этой вершины.

3. Описание алгоритма программы

Для программной реализации алгоритма поиска максимального паросочетания в двудольном графе используются три ключевые структуры данных. Двумерный массив n хранит исходную матрицу смежности двудольного графа размером $size \times size$. Каждый элемент $n[i][j]$ принимает значение 1, если существует ребро между вершиной i левой доли и вершиной j правой доли, и 0 в противном случае. Вектор mt содержит информацию о текущем паросочетании. На начальном этапе вектор mt заполняется значениями -1, что указывает на отсутствие сопоставлений. Если в процессе работы $mt[j]$ принимает значение i , это означает, что вершина j правой доли соединена в паросочетании с вершиной i левой доли. Массив $used$ служит для отметки посещённых вершин левой доли в процессе обхода в глубину и предотвращает повторное рассмотрение вершин в рамках одного поиска увеличивающей цепи.

Основную вычислительную логику алгоритма реализует рекурсивная функция $khun(v)$, выполняющая поиск увеличивающей цепи из вершины v левой доли. Функция возвращает `true`, если увеличивающая цепь найдена, и `false` в противном случае. Работа функции начинается с отметки вершины v как посещённой через присваивание $used[v]=True$. Далее для каждой вершины to правой доли, смежной с v , осуществляется проверка её состояния. Если вершина to свободна ($mt[to]==-1$), происходит немедленное установление соответствия $mt[to]=v$ с возвратом `true`. Если вершина to уже насыщена, функция рекурсивно вызывает себя для вершины левой доли, связанной с to , то есть $khun(mt[to])$. При успешном завершении рекурсивного вызова выполняется чередование паросочетания: $mt[to]=v$, после чего функция также возвращает `true`. Если ни один из вариантов не приводит к успеху, возвращается `false`.

Программа начинается с инициализации пустого паросочетания, заполняя массив mt значениями -1. Матрица смежности n может быть заполнена либо случайными значениями 0 и 1, либо вручную пользователем. После подготовки данных запускается основной цикл поиска максимального паросочетания. Для каждой вершины v левой доли выполняется сброс массива $used$ (все

вершины помечаются как непосещённые) и вызывается функция `khun(v)`. После обработки всех вершин левой доли массив `mt` содержит итоговое максимальное паросочетание. Временная сложность алгоритма составляет $O(n \cdot m)$, где n — количество вершин в одной доле, m — количество рёбер. Пространственная сложность определяется размером матрицы смежности и составляет $O(n^2)$.

Ниже представлен псевдокод функции `create_matrix, initialize_random, initialize_manual, khun, find_max_matching, main`

create_matrix()

1. ФУНКЦИЯ create_matrix(размер):
 2. УСТАНОВИТЬ флаг_создания = ИСТИНА
 3. УСТАНОВИТЬ размер_графа = размер
 4. СОЗДАТЬ матрица[размер][размер]
 5. ДЛЯ i от 0 ДО размер-1:
 6. ДЛЯ j от 0 ДО размер-1:
 - 6.1. матрица[i][j] = 0
 7. ВЕРНУТЬ матрица
8. КОНЕЦ ФУНКЦИИ

initialize_random()

1. ЕСЛИ не gen_create: ВЕРНУТЬ false
2. gen_init $\leftarrow$ true
3. ДЛЯ i=0 ДО size-1:
 4. ДЛЯ j=0 ДО size-1:
 - 4.1. n[i][j] $\leftarrow$ random (0 или 1)
5. ВЕРНУТЬ true

initialize_manual()

1. ЕСЛИ не gen_create: ВЕРНУТЬ false
2. gen_init $\leftarrow$ true
3. ВЫВЕСТИ "Введите граф размером size×size"
4. ДЛЯ i=0 ДО size-1:
 - 4.1. ВЫВЕСТИ "Строка i+1: "
 - 4.2. строка = ВВОД_СТРОКИ ()
 - 4.3. элементы = РАЗДЕЛИТЬ(строка)

5. ЕСЛИ длина(элементы) $\neq$ size:

5.1. ВЫВЕСТИ "Ошибка"

5.2. ВЕРНУТЬ false

6. ДЛЯ j=0 ДО size-1:

6.1. n[i][j] = ЦЕЛОЕ(элементы[j])

7. ВЕРНУТЬ true

khun(v)

1. ЕСЛИ used[v]: ВЕРНУТЬ false

2. used[v] = true

3. ДЛЯ КАЖДОГО соседа to ИЗ graph[v]:

3.1. ЕСЛИ mt[to] = -1 ИЛИ khun(mt[to]):

3.1.1. mt[to] = v

3.1.2. ВЕРНУТЬ true

4. ВЕРНУТЬ false

find_max_matching()

1. ЕСЛИ не (создано И инициализировано):

1.1. ВЕРНУТЬ []

2. создать_список_смежности()

3. mt = [-1] * размер

4. ДЛЯ v=0 ДО размер-1:

4.1. used = [false] * размер

4.2. khun(v)

5. паросочетание = []

6. ДЛЯ i=0 ДО размер-1:

6.1. ЕСЛИ mt[i] $\neq$ -1:

6.1.1. добавить (mt[i], i) В паросочетание

7. ВЕРНУТЬ паросочетание

main():

1. алгоритм = создать_объект_алгоритма_Куна()

2. ПОКА выполнять = истина:

2.1. вывести меню

2.2. ввести выбор пользователя

3. ЕСЛИ выбор = "1":

3.1. запросить размер

3.2. если размер > 2: создать матрицу

4. ИНАЧЕ ЕСЛИ выбор = "2":
 - 4.1. если матрица создана: заполнить случайно
5. ИНАЧЕ ЕСЛИ выбор = "3":
 - 5.1 если матрица создана: заполнить вручную
6. ИНАЧЕ ЕСЛИ выбор = "4":
 - 6.1 если матрица создана и заполнена: вывести
7. ИНАЧЕ ЕСЛИ выбор = "5":
 - 7.1. если матрица создана и заполнена:
 - 7.1.1. найти паросочетание
 - 7.1.2. вывести результат
 - 7.1.3 спросить о сохранении в файл
8. ИНАЧЕ ЕСЛИ выбор = "0":
 - 8.1. выйти из программы
9. ИНАЧЕ:
 - 9.1. сообщить об ошибке выбора

4. Описание программы

Для написания данной программы использован язык программирования Python. Python — это высокоуровневый интерпретируемый язык программирования общего назначения, который завоевал популярность благодаря своей простоте, читаемости кода и богатой экосистеме библиотек. Язык сочетает в себе возможности быстрой разработки приложений и эффективной работы со структурами данных.

Проект разработан в среде разработки PyCharm, которая является одной из наиболее популярных интегрированных сред разработки (IDE) для Python. Программа представляет собой консольное приложение, реализующее алгоритм Куна для поиска максимального паросочетания в двудольном графе.

Программа имеет модульную объектно-ориентированную структуру и состоит из класса `KuhnAlgorithm`, который инкапсулирует всю логику работы с графом и алгоритмом поиска паросочетания, и главной функции `main()`, обеспечивающей взаимодействие с пользователем через консольный интерфейс.

Работа программы начинается с запроса действий пользователя. Если пользователь выбрал “Задать двудольный граф размером $N \times N$ ”, то ему предложат ввести количество вершин в графе. Затем он может выбрать: заполнить массив вручную (с клавиатуры или из файла) или заполнить его автоматически. После заполнения графа пользователь может сразу вызвать алгоритм Куна, который выведет ему наибольшее паросочетание или вывести исходную матрицу смежности на экран.

Часть кода, ответственная за выбор и вызов функций:

```
if choice == "1":
    try:
        size = int(input("Введите количество вершин (больше 2): "))
        if size > 2:
            kuhn.create_matrix(size)
            print(f'Граф размером {size}×{size} создан!')
        else:
            print("Размер должен быть больше 2!")
    except ValueError:
        print("Ошибка: введите целое число!")

elif choice == "2":
```

```

if kuhn.gen_create:
    if kuhn.initialize_random():
        print("Граф заполнен случайными значениями!")
        kuhn.print_matrix()
    else:
        print("Сначала задайте размер графа!")

elif choice == "3":
    if kuhn.gen_create:
        if kuhn.initialize_manual():
            print("Граф успешно введен!")
            kuhn.print_matrix()
        else:
            print("Сначала задайте размер графа!")

```

Часть кода, создающая матрицу смежности, заполняющая её автоматически и записывающая результаты в файл:

```

def create_matrix(self, size):
    """Создание матрицы"""
    self.gen_create = True
    self.size = size
    self.n = [[0] * size for _ in range(size)]
    return self.n

def save_results(self, matching):
    """Сохранение в файл"""
    try:
        with open("kuhn_results.txt", "w", encoding="utf-8") as f:
            f.write("Матрица смежности графа:\n")
            for i in range(self.size):
                for j in range(self.size):
                    f.write(f"{self.n[i][j]:3}")
                f.write("\n")

            f.write("\nНаибольшее паросочетание:\n")
            for pair in matching:
                f.write(f"{pair[0]} - {pair[1]}\n")

            f.write(f"\nВсего найдено {len(matching)} пар\n")
            print("Результаты сохранены в файл 'kuhn_results.txt'")
    except Exception as e:
        print(f"Ошибка при сохранении файла: {e}")

```

Если пользователь захотел ввести матрицу с клавиатуры, то будут задействована следующая функция:

```

def initialize_manual(self):
    """Ручной ввод"""
    if not self.gen_create:
        return False

```

```

self.gen_init = True
print(f"Введите граф размером {self.size} × {self.size} (по строкам, только 0 и 1):")
for i in range(self.size):
    row = input(f"Строка {i + 1} ({self.size} чисел через пробел): ").split()
    if len(row) != self.size:
        print(f"Ошибка: нужно ввести ровно {self.size} чисел")
        return False
    for j in range(self.size):
        self.n[i][j] = int(row[j])
return True

```

После ввода двудольного графа, пользователь может вывести его на экран:

```

def print_matrix(self):
    """Вывод матрицы"""
    if not (self.gen_create and self.gen_init):
        return False

    print("\nМатрица смежности графа:")
    for i in range(self.size):
        for j in range(self.size):
            print(f"{self.n[i][j]:3}", end="")
        print()
    return True

```

Вызвать алгоритм Куна для поиска наибольшего паросочетания:
 Часть кода, отвечающая за вызов:

```

elif choice == "5":
    matching = kuhn.find_max_matching()
    if matching:
        print("\n" + "=" * 50)
        print("НАИБОЛЬШЕЕ ПАРОСОЧЕТАНИЕ:")
        print("=" * 50)

        kuhn.print_matrix()

        print(f"\nНайдено пар: {len(matching)}")
        print("Паросочетания (левая часть - правая часть):")
        for pair in matching:
            print(f" {pair[0]} → {pair[1]}")

        # Сохранение результатов
        save = input("\nСохранить результаты в файл? (да/нет): ").strip().lower()
        if save in ['да', 'yes', 'y', 'д']:
            kuhn.save_results(matching)
    else:
        print("Сначала создайте и заполните граф!")

```

Часть кода, отвечающая за алгоритм.

```
def khun(self, v):  
    """Рекурсивная часть алгоритма Куна"""  
    if self.used[v]:  
        return False  
  
    self.used[v] = True  
    for to in self.graph[v]:  
        if self.mt[to] == -1 or self.khun(self.mt[to]):  
            self.mt[to] = v  
            return True  
    return False
```

Ниже можно увидеть оформление начального запроса и дальнейшие действия с ним.

```
=====
АЛГОРИТМ КУНА ДЛЯ ПОИСКА НАИБОЛЬШЕГО ПАРОСОЧЕТАНИЯ
=====
1. Задать размер графа (N×N)
2. Заполнить граф случайными значениями
3. Заполнить граф вручную
4. Вывести матрицу графа
5. Найти наибольшее паросочетание (алгоритм Куна)
0. Выход
-----
```

Рисунок 2 – Стартовое окно программы.

Генерация графа размером 5 вершин, его автоматическое заполнение и

В
Ы
В
О
Д
Н
а
Э
К
р
а
н
.

```
Введите количество вершин (больше 2): 5
Граф размером 5×5 создан!

=====
АЛГОРИТМ КУНА ДЛЯ ПОИСКА НАИБОЛЬШЕГО ПАРОСОЧЕТАНИЯ
=====
1. Задать размер графа (N×N)
2. Заполнить граф случайными значениями
3. Заполнить граф вручную
4. Вывести матрицу графа
5. Найти наибольшее паросочетание (алгоритм Куна)
0. Выход
-----

Выберите действие: 2
Граф заполнен случайными значениями!

Матрица смежности графа:
  0  1  1  0  1
  0  1  1  1  0
  1  1  1  0  1
  1  1  0  1  1
  0  1  0  1  1
```

Рисунок 3-Случайная генерация.

Вызов функции поиска наибольшего паросочетания и вывод его на экран:

```
Выберите действие: 5

=====
НАИБОЛЬШЕЕ ПАРОСОЧЕТАНИЕ:
=====

Матрица смежности графа:
  0  1  1  0  1
  0  1  1  1  0
  1  1  1  0  1
  1  1  0  1  1
  0  1  0  1  1

Найдено пар: 5
Паросочетания (левая часть - правая часть):
  2 → 0
  4 → 1
  1 → 2
  3 → 3
  0 → 4
```

Рисунок 4 – Алгоритм Куна.

5. Тестирование

Среда разработки PyCharm предоставляет комплексные средства для разработки, отладки и тестирования программ на языке Python.

Тестирование проводилось в рабочем порядке, в процессе разработки и после завершения написания программы. В ходе тестирования было выявлено и исправлено множество проблем, связанных с вводом и обращением к данным, изменением формата используемых данных, алгоритмом программы, взаимодействием функций.

Ниже продемонстрирован результат тестирования программы при вводе пользователем различных количеств вершин и вызове функций без предварительного создания графа.

Тестирование на ввод вершин:

```
=====
АЛГОРИТМ КУНА ДЛЯ ПОИСКА НАИБОЛЬШЕГО ПАРОСОЧЕТАНИЯ
=====
1. Задать размер графа (N×N)
2. Заполнить граф случайными значениями
3. Заполнить граф вручную
4. Вывести матрицу графа
5. Найти наибольшее паросочетание (алгоритм Куна)
0. Выход
-----
Выберите действие: 1
Введите количество вершин (больше 2): 4
Граф размером 4×4 создан!
```

Рисунок 5 – Тестирование при вводе количества вершин – 4.

Тестирование на ввод одной вершины:

```
=====
АЛГОРИТМ КУНА ДЛЯ ПОИСКА НАИБОЛЬШЕГО ПАРОСОЧЕТАНИЯ
=====

1. Задать размер графа (N×N)
2. Заполнить граф случайными значениями
3. Заполнить граф вручную
4. Вывести матрицу графа
5. Найти наибольшее паросочетание (алгоритм Куна)
0. Выход

-----

Выберите действие: 1
Введите количество вершин (больше 2): 1
Размер должен быть больше 2!
```

Рисунок 6 – Тестирование при вводе количества вершин – 1.

Тестирование на автоматическую генерацию графа и его вывод на экран:

```
=====
АЛГОРИТМ КУНА ДЛЯ ПОИСКА НАИБОЛЬШЕГО ПАРОСОЧЕТАНИЯ
=====

1. Задать размер графа (N×N)
2. Заполнить граф случайными значениями
3. Заполнить граф вручную
4. Вывести матрицу графа
5. Найти наибольшее паросочетание (алгоритм Куна)
0. Выход

-----

Выберите действие: 2
Граф заполнен случайными значениями!

Матрица смежности графа:
  1  1  1  0
  0  1  0  1
  0  0  1  0
  1  1  0  1
```

Рисунок 7 – Тестирование на вывод матрицы.

Тестирование на автоматическое заполнение графа без его создания:

```
=====
АЛГОРИТМ КУНА ДЛЯ ПОИСКА НАИБОЛЬШЕГО ПАРОСОЧЕТАНИЯ
=====
1. Задать размер графа (N×N)
2. Заполнить граф случайными значениями
3. Заполнить граф вручную
4. Вывести матрицу графа
5. Найти наибольшее паросочетание (алгоритм Куна)
0. Выход
-----
Выберите действие: 2
Сначала задайте размер графа!
```

Рисунок 8 - Тестирование на автомат. заполнение без создания.

Тестирование на ручное заполнение графа с клавиатуры без его создания:

```
=====
АЛГОРИТМ КУНА ДЛЯ ПОИСКА НАИБОЛЬШЕГО ПАРОСОЧЕТАНИЯ
=====
1. Задать размер графа (N×N)
2. Заполнить граф случайными значениями
3. Заполнить граф вручную
4. Вывести матрицу графа
5. Найти наибольшее паросочетание (алгоритм Куна)
0. Выход
-----
Выберите действие: 3
Сначала задайте размер графа!
```

Рисунок 9 - Тестирование на ручное заполнение без создания.

Тестирование на вывод графа без его создания:

```
=====
АЛГОРИТМ КУНА ДЛЯ ПОИСКА НАИБОЛЬШЕГО ПАРОСОЧЕТАНИЯ
=====
1. Задать размер графа (N×N)
2. Заполнить граф случайными значениями
3. Заполнить граф вручную
4. Вывести матрицу графа
5. Найти наибольшее паросочетание (алгоритм Куна)
0. Выход
-----
Выберите действие: 4
Сначала создайте и заполните граф!
```

Рисунок 10 - Тестирование на вывод без создания.

Тестирование на вызов алгоритма Куна без создания графа:

```
=====
АЛГОРИТМ КУНА ДЛЯ ПОИСКА НАИБОЛЬШЕГО ПАРОСОЧЕТАНИЯ
=====
1. Задать размер графа (N×N)
2. Заполнить граф случайными значениями
3. Заполнить граф вручную
4. Вывести матрицу графа
5. Найти наибольшее паросочетание (алгоритм Куна)
0. Выход
-----
Выберите действие: 5
Сначала создайте и заполните граф!
```

Рисунок 11 - Тестирование на вызов алгоритма без создания.

Тестирование на ручной ввод графа с клавиатуры:

```
=====
АЛГОРИТМ КУНА ДЛЯ ПОИСКА НАИБОЛЬШЕГО ПАРОСОЧЕТАНИЯ
=====

1. Задать размер графа (N×N)
2. Заполнить граф случайными значениями
3. Заполнить граф вручную
4. Вывести матрицу графа
5. Найти наибольшее паросочетание (алгоритм Куна)
0. Выход

-----
Выберите действие: 3
Введите граф размером 3×3 (по строкам, только 0 и 1):
Строка 1 (3 чисел через пробел): 1 0 1
Строка 2 (3 чисел через пробел): 0 0 1
Строка 3 (3 чисел через пробел): 1 0 0
Граф успешно введен!

Матрица смежности графа:
  1  0  1
  0  0  1
  1  0  0
```

Рисунок 12 - Тестирование на ввод графа с клавиатуры.

Тестирование на работу алгоритма Куна:

```
=====
АЛГОРИТМ КУНА ДЛЯ ПОИСКА НАИБОЛЬШЕГО ПАРОСОЧЕТАНИЯ
=====
1. Задать размер графа (N×N)
2. Заполнить граф случайными значениями
3. Заполнить граф вручную
4. Вывести матрицу графа
5. Найти наибольшее паросочетание (алгоритм Куна)
0. Выход
-----
Выберите действие: 5

=====
НАИБОЛЬШЕЕ ПАРОСОЧЕТАНИЕ:
=====

Матрица смежности графа:
  1  0  1
  0  0  1
  1  0  0

Найдено пар: 2
Паросочетания (левая часть - правая часть):
  0 → 0
  1 → 2
```

Рисунок 13 - Тестирование на работу алгоритма Куна.

Таблица 1 – Описание поведения программы при тестировании.

Описание теста	Ожидаемый результат	Полученный результат
Запуск программы	Вывод пользователю списка с функциями программы	Верно
Задание размера матрицы	Вывод пользователю предложения задать размер матрицы числом	Верно

Задание матрицы единичной вершиной	Вывод пользователю предупреждения о не-правильности вводимых данных	Верно
Тестирование на вывод матрицы	Генерация и вывод матрицы на экран	Верно
Вызов функции автоматического заполнения без задания размера графа.	Вывод пользователю предупреждения о необходимости создания графа	Верно
Вызов функции ручного заполнения с клавиатуры без задания размера графа.	Вывод пользователю предупреждения о необходимости задания размера графа	Верно
Вызов функции вывода графа без его создания	Вывод пользователю предупреждения о необходимости создания графа	Верно
Вызов функции алгоритма Куна без создания графа	Вывод пользователю предупреждения о необходимости создания графа	Верно
Тестирование на ручной ввод графа с клавиатуры	Запись графа в массив	Верно
Тестирование на работу алгоритма Куна.	Вывод результата работы алгоритма на экран	Верно

Заключение

Таким образом, в процессе создания данного проекта разработана программа, реализующая алгоритм Куна для поиска наибольшего паросочетания в среде разработки PyCharm.

При выполнении данной курсовой работы были получены практические навыки разработки программ на языке Python и освоены методы работы с двудольными графами и матричными структурами данных. Приобретён опыт реализации алгоритма Куна для решения задачи о максимальном паросочетании. Углублены знания языка программирования Python и особенностей объектно-ориентированного программирования.

Недостатком разработанной программы является консольный характер пользовательского интерфейса. Однако это позволило сосредоточиться на реализации алгоритмической составляющей, избежав излишней сложности графического интерфейса. Консольный режим обеспечивает лёгкость переносимости программы между различными операционными системами.

Программный код написан с учётом принципов читаемости и модульности, содержит подробные комментарии и обработку исключительных ситуаций. Интерфейс программы интуитивно понятен и удобен для использования как в учебных, так и в практических целях.

Список литературы

1. Эрик Мэтиз. Изучаем Python: программирование игр, визуализация данных, веб-приложения.
2. В. Л. Дольников О. П. Якимова. Основные алгоритмы на графах. 2011 г.
3. Уилсон Р. Введение в теорию графов. Пер. с анг. 1977. 208 с.

Приложение А.

Листинг программы.

```
import random

class KuhnAlgorithm:
    def __init__(self):
        self.size = 0
        self.graph = []
        self.n = []
        self.mt = []
        self.used = []
        self.gen_create = False
        self.gen_init = False

    def create_matrix(self, size):
        """Создание матрицы"""
        self.gen_create = True
        self.size = size
        self.n = [[0] * size for _ in range(size)]
        return self.n

    def initialize_random(self):
        """Заполнение матрицы 0 или 1"""
        if not self.gen_create:
            return False

        self.gen_init = True
        for i in range(self.size):
            for j in range(self.size):
                self.n[i][j] = random.randint(0, 1)
        return True

    def initialize_manual(self):
        """Ручной ввод"""
        if not self.gen_create:
            return False

        self.gen_init = True
        print(f"Введите граф размером {self.size}×{self.size} (по строкам, только 0 и 1):")
        for i in range(self.size):
            row = input(f"Строка {i + 1} ({self.size} чисел через пробел):")
            row = row.split()
            if len(row) != self.size:
                print(f"Ошибка: нужно ввести ровно {self.size} чисел")
                return False
            for j in range(self.size):
                self.n[i][j] = int(row[j])
        return True

    def print_matrix(self):
        """Вывод матрицы"""
        if not (self.gen_create and self.gen_init):
            return False

        print("\nМатрица смежности графа:")
        for i in range(self.size):
            for j in range(self.size):
                print(f"{self.n[i][j]:3}", end=" ")
            print()
```



```

        return True

def generate_adjacency_list(self):
    """Генерация смежности"""
    self.graph = [[] for _ in range(self.size)]
    for i in range(self.size):
        for j in range(self.size):
            if self.n[i][j] != 0:
                self.graph[i].append(j)

def khun(self, v):
    """Рекурсивная часть алгоритма Куна"""
    if self.used[v]:
        return False

    self.used[v] = True
    for to in self.graph[v]:
        if self.mt[to] == -1 or self.khun(self.mt[to]):
            self.mt[to] = v
            return True
    return False

def find_max_matching(self):
    """Поиск наибольшего паросочетания"""
    if not (self.gen_create and self.gen_init):
        return []

    self.generate_adjacency_list()
    self.mt = [-1] * self.size

    for v in range(self.size):
        self.used = [False] * self.size
        self.khun(v)

    # Формируем список пар паросочетания
    matching = []
    for i in range(self.size):
        if self.mt[i] != -1:
            matching.append((self.mt[i], i))

    return matching

def save_results(self, matching):
    """Сохранение в файл"""
    try:
        with open("kuhn_results.txt", "w", encoding="utf-8") as f:
            f.write("Матрица смежности графа:\n")
            for i in range(self.size):
                for j in range(self.size):
                    f.write(f"{self.n[i][j]:3}")
                f.write("\n")

            f.write("\nНаибольшее паросочетание:\n")
            for pair in matching:
                f.write(f"{pair[0]} - {pair[1]}\n")

            f.write(f"\nВсего найдено {len(matching)} пар\n")
            print("Результаты сохранены в файл 'kuhn_results.txt'")
    except Exception as e:
        print(f"Ошибка при сохранении файла: {e}")

def main():

```

```

kuhn = KuhnAlgorithm()

while True:
    print("\n" + "=" * 50)
    print("АЛГОРИТМ КУНА ДЛЯ ПОИСКА НАИБОЛЬШЕГО ПАРОСОЧЕТАНИЯ")
    print("=" * 50)
    print("1. Задать размер графа (N×N)")
    print("2. Заполнить граф случайными значениями")
    print("3. Заполнить граф вручную")
    print("4. Вывести матрицу графа")
    print("5. Найти наибольшее паросочетание (алгоритм Куна)")
    print("0. Выход")
    print("-" * 50)

    try:
        choice = input("Выберите действие: ").strip()

        if choice == "1":
            try:
                size = int(input("Введите количество вершин (больше 2): "))

                if size > 2:
                    kuhn.create_matrix(size)
                    print(f"Граф размером {size}×{size} создан!")
                else:
                    print("Размер должен быть больше 2!")
            except ValueError:
                print("Ошибка: введите целое число!")

        elif choice == "2":
            if kuhn.gen_create:
                if kuhn.initialize_random():
                    print("Граф заполнен случайными значениями!")
                    kuhn.print_matrix()
            else:
                print("Сначала задайте размер графа!")

        elif choice == "3":
            if kuhn.gen_create:
                if kuhn.initialize_manual():
                    print("Граф успешно введен!")
                    kuhn.print_matrix()
            else:
                print("Сначала задайте размер графа!")

        elif choice == "4":
            if not kuhn.print_matrix():
                print("Сначала создайте и заполните граф!")

        elif choice == "5":
            matching = kuhn.find_max_matching()
            if matching:
                print("\n" + "=" * 50)
                print("НАИБОЛЬШЕЕ ПАРОСОЧЕТАНИЕ:")
                print("=" * 50)

                kuhn.print_matrix()

                print(f"\nНайдено пар: {len(matching)}")
                print("Паросочетания (левая часть - правая часть):")
                for pair in matching:
                    print(f"    {pair[0]} → {pair[1]}")
            
```

```
        # Сохранение результатов
        save = input("\nСохранить результаты в файл? (да/нет):")
    ").strip().lower()
        if save in ['да', 'yes', 'y', 'д']:
            kuhn.save_results(matching)
        else:
            print("Сначала создайте и заполните граф!")

    elif choice == "0":
        print("Выход из программы...")
        break

    else:
        print("Неверный выбор! Попробуйте снова.")

except KeyboardInterrupt:
    print("\nПрограмма прервана пользователем.")
    break
except Exception as e:
    print(f"Произошла ошибка: {e}")

if __name__ == "__main__":
    main()
```